

Chapter 15. The Page Cache

As already mentioned in the section "[The Common File Model](#)" in [Chapter 12](#), a disk cache is a software mechanism that allows the system to keep in RAM some data that is normally stored on a disk, so that further accesses to that data can be satisfied quickly without accessing the disk.

Disk caches are crucial for system performance, because repeated accesses to the same disk data are quite common. A User Mode process that interacts with a disk is entitled to ask repeatedly to read or write the same disk data. Moreover, different processes may also need to address the same disk data at different times. As an example, you may use the `cp` command to copy a text file and then invoke your favorite editor to modify it. To satisfy your requests, the command shell will create two different processes that access the same file at different times.

We have already encountered other disk caches in [Chapter 12](#): the dentry cache, which stores dentry objects representing filesystem pathnames, and the inode cache, which stores inode objects representing disk inodes. Notice, however, that dentry objects and inode objects are not mere buffers storing the contents of some disk blocks; thus, the dentry cache and the inode cache are rather peculiar as disk caches.

This chapter deals with the page cache, which is a disk cache working on whole pages of data. We introduce the page cache in the first section. Then, we discuss in the section "[Storing Blocks in the Page Cache](#)" how the page cache can be used to retrieve single blocks of data (for instance, superblocks and inodes); this feature is crucial to speed up the VFS and the disk-based filesystems. Next, we describe in the section "[Writing Dirty Pages to Disk](#)" how the dirty pages in the page cache are written back to disk. Finally, we mention in the last section "[The `sync\(\)`, `fsync\(\)`, and `fdatasync\(\)` System Calls](#)" some system calls that allow a user to flush the contents of the page cache so as to update the disk contents.

15.1. The Page Cache

The page cache is the main disk cache used by the Linux kernel. In most cases, the kernel refers to the page cache when reading from or writing to disk. New pages are added to the page cache to satisfy User Mode processes's read requests. If the page is not already in the cache, a new entry is added to the cache and filled with the data read from the disk. If there is enough free memory, the page is kept in the cache for an indefinite period of time and can then be reused by other processes without accessing the disk.

Similarly, before writing a page of data to a block device, the kernel verifies whether the corresponding page is already included in the cache; if not, a new entry is added to the cache and filled with the data to be written on disk. The I/O data transfer does not start immediately: the disk update is delayed for a few seconds, thus giving a chance to the processes to further modify the data to be written (in other words, the kernel implements deferred write operations).

Kernel code and kernel data structures don't need to be read from or written to disk.^[*] Thus, the pages included in the page cache can be of the following types:

[*] Well, almost never: if you want to resume the whole state of the system after a shutdown, you can perform a "suspend to disk" operation (hibernation), which saves the content of the whole RAM on a swap partition. We won't further discuss this case.

- Pages containing data of regular files; in [Chapter 16](#), we describe how the kernel handles read, write, and memory mapping operations on them.
- Pages containing directories; as we'll see in [Chapter 18](#), Linux handles the directories much like regular files.
- Pages containing data directly read from block device files (skipping the filesystem layer); as discussed in [Chapter 16](#), the kernel handles them using the same set of functions as for pages containing data of regular files.
- Pages containing data of User Mode processes that have been swapped out on disk. As we'll see in [Chapter 17](#), the kernel could be forced to keep in the page cache some pages whose contents have been already written on a swap area (either a regular file or a disk partition).

- Pages belonging to files of special filesystems, such as the shm special filesystem used for Interprocess Communication (IPC) shared memory region (see [Chapter 19](#)).

As you can see, each page included in the page cache contains data belonging to some file. This file or more precisely the file's inode is called the page's owner. (As we will see in [Chapter 17](#), pages containing swapped-out data have the same owner even if they refer to different swap areas.)

Practically all `read( )` and `write( )` file operations rely on the page cache. The only exception occurs when a process opens a file with the `O_DIRECT` flag set: in this case, the page cache is bypassed and the I/O data transfers make use of buffers in the User Mode address space of the process (see the section "Direct I/O Transfers" in [Chapter 16](#)); several database applications make use of the `O_DIRECT` flag so that they can use their own disk caching algorithm.

Kernel designers have implemented the page cache to fulfill two main requirements:

- Quickly locate a specific page containing data relative to a given owner. To take the maximum advantage from the page cache, searching it should be a very fast operation.
- Keep track of how every page in the cache should be handled when reading or writing its content. For instance, reading a page from a regular file, a block device file, or a swap area must be performed in different ways, thus the kernel must select the proper operation depending on the page's owner.

The unit of information kept in the page cache is, of course, a whole page of data. As we'll see in [Chapter 18](#), a page does not necessarily contain physically adjacent disk blocks, so it cannot be identified by a device number and a block number. Instead, a page in the page cache is identified by an owner and by an index within the owner's data; usually, an inode and an offset inside the corresponding file.

15.1.1. The `address_space` Object

The core data structure of the page cache is the `address_space` object, a data structure embedded in the inode object that owns the page.^[*] Many pages in the cache may refer to the same owner, thus they may be linked to the

same `address_space` object. This object also establishes a link between the owner's pages and a set of methods that operate on these pages.

[*] An exception occurs for pages that have been swapped out. As we will see in [Chapter 17](#), these pages have a common `address_space` object not included in any inode.

Each page descriptor includes two fields called `mapping` and `index`, which link the page to the page cache (see the section "[Page Descriptors](#)" in [Chapter 8](#)). The first field points to the `address_space` object of the inode that owns the page. The second field specifies the offset in page-size units within the owner's "address space," that is, the position of the page's data inside the owner's disk image. These two fields are used when looking for a page in the page cache.

Quite surprisingly, the page cache may happily contain multiple copies of the same disk data. For instance, the same 4-KB block of data of a regular file can be accessed in the following ways:

- Reading the file; hence, the data is included in a page owned by the regular file's inode.
- Reading the block from the device file (disk partition) that hosts the file; hence, the data is included in a page owned by the master inode of the block device file.

Thus, the same disk data appears in two different pages referenced by two different `address_space` objects.

The fields of the `address_space` object are shown in [Table 15-1](#).

Table 15-1. The fields of the `address_space` object

Type	Field	Description
<code>struct inode *</code>	<code>host</code>	Pointer to the inode hosting this object, if any
<code>struct radix_tree_root</code>	<code>page_tree</code>	Root of radix tree identifying the owner's pages
<code>spinlock_t</code>	<code>tree_lock</code>	Spin lock protecting the radix tree
<code>unsigned int</code>	<code>i_mmap_writable</code>	Number of shared memory mappings in the address space
<code>struct prio_tree_root</code>	<code>i_mmap</code>	Root of the radix priority search tree (see Chapter 17)

<code>struct list_head</code>	<code>i_mmap_nonlinear</code>	List of non-linear memory regions in the address space
<code>spinlock_t</code>	<code>i_mmap_lock</code>	Spin lock protecting the radix priority search tree
<code>unsigned int</code>	<code>TRuncate_count</code>	Sequence counter used when truncating the file
<code>unsigned long</code>	<code>nrpages</code>	Total number of owner's pages
<code>unsigned long</code>	<code>writeback_index</code>	Page index of the last write-back operation on the owner's pages
<code>struct address_space_operations *</code>	<code>a_ops</code>	Methods that operate on the owner's pages
<code>unsigned long</code>	<code>flags</code>	Error bits and memory allocator flags
<code>struct backing_dev_info *</code>	<code>backing_dev_info</code>	Pointer to the <code>backing_dev_info</code> of the block device holding the data of this owner
<code>spinlock_t</code>	<code>private_lock</code>	Usually, spin lock used when managing the <code>private_list</code> list
<code>struct list head</code>	<code>private_list</code>	Usually, a list of dirty buffers of indirect blocks associated with the inode
<code>struct address_space *</code>	<code>assoc_mapping</code>	Usually, pointer to the <code>address_space</code> object of the block device including the indirect blocks

If the owner of a page in the page cache is a file, the `address_space` object is embedded in the `i_data` field of a VFS inode object. The `i_mapping` field of the inode always points to the `address_space` object of the owner of the pages containing the inode's data. The `host` field of the `address_space` object points to the inode object in which the descriptor is embedded.

Thus, if a page belongs to a file that is stored in an Ext3 filesystem, the owner of the page is the inode of the file and the corresponding `address_space` object is stored in the `i_data` field of the VFS inode object. The `i_mapping` field of the inode points to the `i_data` field of the same inode, and the `host` field of the `address_space` object points to the same inode.

Sometimes, however, things are more complicated. If a page contains data read from a block device file that is, it stores "raw" data of a block device the `address_space` object is embedded in the "master" inode of the file in the `bdev` special filesystem associated with the block device (this inode is referenced by the `bd_inode` field of the block device descriptor, see the section "[Block Devices](#)" in [Chapter 14](#)). Therefore, the `i_mapping` field of an inode of a block device file points to the `address_space` object embedded in the master inode; correspondingly, the `host` field of the `address_space` object points to the master inode. In this way, all pages containing data read from a block device have the same `address_space` object, even if they have been accessed by referring to different block device files.

The `i_mmap`, `i_mmap_writable`, `i_mmap_nonlinear`, and `i_mmap_lock` fields refer to memory mapping and reverse mapping. We'll discuss these topics in [Chapter 16](#) and [Chapter 17](#).

The `backing_dev_info` field points the `backing_dev_info` descriptor associated with the block device storing the data of the owner. As explained in the section "[Request Queue Descriptors](#)" in [Chapter 14](#), the `backing_dev_info` structure is usually embedded in the request queue descriptor of the block device.

The `private_list` field is the head of a generic list that can be freely used by the filesystem for its specific purposes. For example, the Ext2 filesystem makes use of this list to collect the dirty buffers of "indirect" blocks associated with the inode (see the section "[Data Blocks Addressing](#)" in [Chapter 18](#)). When a flush operation forces the inode to be written to disk, the kernel flushes also all the buffers in this list. Moreover, the Ext2 filesystem stores in the `assoc_mapping` field a pointer to the `address_space` object of the block device containing the indirect blocks; it also uses the `assoc_mapping->private_lock` spin lock to protect the lists of indirect blocks in multiprocessor systems.

A crucial field of the `address_space` object is `a_ops`, which points to a table of type `address_space_operations` containing the methods that define how the owner's pages are handled. These methods are shown in [Table 15-2](#).

Table 15-2. The methods of the `address_space` object

Method	Description
<code>writepage</code>	Write operation (from the page to the owner's disk image)
<code>readpage</code>	Read operation (from the owner's disk image to the page)
<code>sync_page</code>	Start the I/O data transfer of already scheduled operations on owner's pages
<code>writepages</code>	Write back to disk a given number of dirty owner's pages
<code>set_page_dirty</code>	Set an owner's page as dirty
<code>readpages</code>	Read a list of owner's pages from disk
<code>prepare_write</code>	Prepare a write operation (used by disk-based filesystems)

`commit_write`

Complete a write operation (used by disk-based filesystems)

`bmap`

Get a logical block number from a file block index

`invalidatepage`

Invalidate owner's pages (used when truncating the file)

`releasepage`

Used by journaling filesystems to prepare the release of a page

`direct_IO`

Direct I/O transfer of the owner's pages (bypassing the page cache)

The most important methods are `readpage`, `writpage`, `prepare_write`, and `commit_write`. We discuss them in [Chapter 16](#). In most cases, the methods link the owner inode objects with the low-level drivers that access the physical devices. For instance, the function that implements the `readpage` method for an inode of a regular file knows how to locate the positions on the physical disk device of the blocks corresponding to each page of the file. In this chapter, however, we don't have to discuss the `address_space` methods further.

15.1.2. The Radix Tree

In Linux, files can have large sizes, even a few terabytes. When accessing a large file, the page cache may become filled with so many of the file's pages that sequentially scanning all of them would be too time-consuming. In order to perform page cache lookup efficiently, Linux 2.6 makes use of a large set of search trees, one for each `address_space` object.

The `page_tree` field of an `address_space` object is the root of a radix tree, which contains pointers to the descriptors of the owner's pages. Given a page index denoting the position of the page inside the owner's disk image, the kernel can perform a very fast lookup operation in order to determine whether the required page is already included in the page cache. When looking up the page, the kernel interprets the index as a path inside the radix tree and quickly reaches the position where the page descriptor is or should be stored. If found, the kernel can retrieve from the radix tree the descriptor of the page; it can also quickly determine whether the page is dirty (i.e., to be flushed to disk) and whether an I/O transfer for its data is currently on-going.

Each node of the radix tree can have up to 64 pointers to other nodes or to page descriptors. Nodes at the bottom level store pointers to page descriptors (the leaves), while nodes at higher levels store pointers to other nodes (the children). Each node is represented by the `radix_tree_node` data structure, which includes three fields: `slots` is an array of 64 pointers, `count` is a counter of how many pointers in the node are not NULL, and `tags` is a two-component array of flags that will be discussed in the section ["The Tags of the Radix Tree"](#) later in this chapter. The root of the tree is represented by a `radix_tree_root` data structure, having three fields: `height` denotes the current tree's height (number of levels excluding the leaves),

`gfp_mask` specifies the flags used when requesting memory for a new node, and `rnode` points to the `radix_tree_node` data structure corresponding to the node at level 1 of the tree (if any).

Let us consider a simple example. If none of the indices stored in the tree is greater than 63, the tree height is equal to one, because the 64 potential leaves can all be stored in the node at level 1 (see Figure 15-1 (a)). If, however, a new page descriptor corresponding to index 131 must be stored in the page cache, the tree height is increased to two, so that the radix tree can pinpoint indices up to 4095 (see Figure 15-1(b)).

Figure 15-1. Two examples of a radix tree

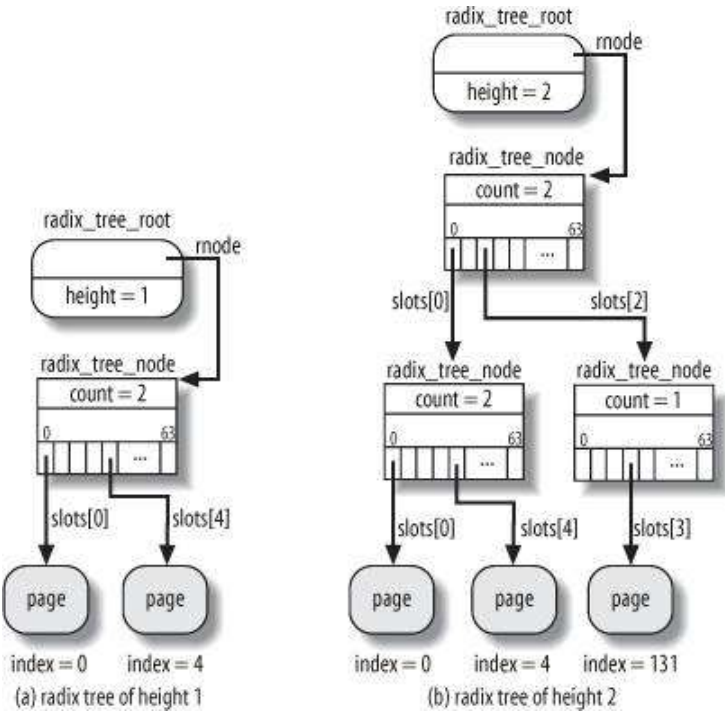


Table 15-3 shows the highest page index and the corresponding maximum file size for each given height of the radix tree on a 32-bit architecture. In this case, the maximum height of a radix tree is six, although it is quite unlikely that the page cache of your system will make use of a radix tree that huge. Because the page index is stored in a 32-bit variable, when the tree has height equal to six, the node at the highest level can have at most four children.

Table 15-3. Highest index and maximum file size for each radix tree height

Radix tree height

Highest index

Maximum file size

0

none

0 bytes

8

1

$$2^6 - 1 = 63$$

256 kilobytes

2

$$2^{12} - 1 = 4\,095$$

16 megabytes

3

$$2^{18} - 1 = 262\,143$$

1 gigabyte

4

$$2^{24} - 1 = 16\,777\,215$$

64 gigabytes

5

$$2^{30} - 1 = 1\,073\,741\,823$$

4 terabytes

6

$$2^{32} - 1 = 4\,294\,967\,295$$

16 terabytes

The best way to understand how page lookup is performed is to recall how the paging system makes use of the page tables to translate linear addresses into physical addresses. As discussed in the section "[Regular Paging](#)" in [Chapter 2](#), the 20 most significant bits of a linear address are split into two 10-bit long fields: the first field is an offset in the Page Directory, while the second one is an offset in the Page Table pointed to by the proper Page Directory entry.

A similar approach is used in the radix tree. The equivalent of the linear address is the page's index. However, the number of fields to be considered in the page's index depends on the height of the radix tree. If the radix tree has height 1, only indices ranging from 0 to 63 can be represented, thus the 6 less significant bits of the page's index are interpreted as the `slots` array index for the single node at level 1. If the radix tree has height 2, the indices that can be represented range from 0 to 4095; the 12 less significant bits of the page's index are thus split in 2 fields of 6 bits each; the most significant field is used as an array index for the node at level 1, while the less significant field is used as an array index for the node at level 2. The procedure is similar for every other radix tree's height. If the height is equal to 6, the 2 most significant bits of the page's index are the array index for the node at level 1, the following 6 bits are the array index for the node at level 2, and so on up to the 6 less significant bits, which are the array index for the node at level 6.

8

If the highest index of a radix tree is smaller than the index of a page that should be added, then the kernel increases the tree height correspondingly; the intermediate nodes of the radix tree depend on the value of the page index (see [Figure 15-1](#) for an example).

15.1.3. Page Cache Handling Functions

The basic high-level functions that use the page cache involve finding, adding, and removing a page. Another function based on the previous ones ensures that the cache includes an up-to-date version of a given page.

15.1.3.1. Finding a page

The `find_get_page( )` function receives as its parameters a pointer to an `address_space` object and an offset value. It acquires the address space's spin lock and invokes the `radix_tree_lookup( )` function to search for a leaf node of the radix tree having the required offset. This function, in turn, starts from the root node of the tree and goes down according to the bits of the offset value, as explained in the previous section. If a `NULL` pointer is encountered, the function returns `NULL`; otherwise, it returns the address of a leaf node, that is, the pointer of the required page descriptor. If the requested page is found, `find_get_page( )` increases its usage counter, releases the spin lock, and returns its address; otherwise, the function releases the spin lock and returns `NULL`.

The `find_get_pages( )` function is similar to `find_get_page( )`, but it performs a page cache lookup for a group of pages having contiguous indices. It receives as its parameters a pointer to an `address_space` object, the offset in the address space from where to start searching, the maximum number of pages to be retrieved, and a pointer to an array of pages descriptors to be filled by the function. To perform the lookup operation, `find_get_pages( )` relies on the `radix_tree_gang_lookup( )` function, which fills the array of pointers and returns the number of pages found. The returned pages have ascending indices, although there may be holes in the indices because some pages may not be in the page cache.

There are several other functions that perform search operations on the page cache. For example, the `find_lock_page( )` function is similar to `find_get_page( )`, but it increases the usage counter of the returned page and invokes `lock_page( )` to set the `PG_locked` flag; thus, when the function returns, the page can be accessed exclusively by the caller. The `lock_page( )` function, in turn, blocks the current process if the page is already locked. To that end, it invokes the `__wait_on_bit_lock( )` function on the `PG_locked` bit. The latter function puts the current process in the `TASK_UNINTERRUPTIBLE` state, stores the process descriptor in a wait queue, executes the `sync_page` method of the `address_space` object to unplug the request queue of the block device containing the file, and finally invokes `schedule( )` to suspend the process until the `PG_locked` flag of the page is cleared. To unlock a page and wake up the processes sleeping in the wait queue, the kernel makes use of the `unlock_page( )` function.

The `find_trylock_page( )` function is similar to `find_lock_page( )`, except that it never blocks: if the requested page is already locked, the function returns an error code. Finally, the `find_or_create_page( )` function executes `find_lock_page( )`; however, if the page is not found, a new page is allocated and inserted in the page cache.

15.1.3.2. Adding a page

The `add_to_page_cache( )` function inserts a new page descriptor in the page cache. It receives as its parameters the address page of the page descriptor, the address mapping of an `address_space` object, the value `offset` representing the page index inside the address space, and the memory allocation flags `gfp_mask` to be used when allocating the new nodes of the radix tree. The function performs the following

operations:

1. Invokes `radix_tree_preload( )`, which disables kernel preemption and fills the per-CPU variable `radix_tree_preloads` with a few free `radix_tree_node` structures. Allocation of `radix_tree_node` structures is done by means of the `radix_tree_node_cachep` slab allocator cache. If `radix_tree_preload( )` fails in preallocating the `radix_tree_node` structures, the `add_to_page_cache( )` function terminates by returning the error code `-ENOMEM`. Otherwise, if `radix_tree_preload( )` succeeds, `add_to_page_cache( )` can be sure that the insertion of the new page descriptor will not fail for lack of free memory, at least for files of size up to 64 GB.
2. Acquires the `mapping->tree_lock` spin lock notice that kernel preemption has already been disabled by `radix_tree_preload( )`.
3. Invokes `radix_tree_insert( )` to insert the new node in the tree. This function performs the following steps:
 - a. Invokes `radix_tree_maxindex( )` to get the maximum index that can be inserted in the radix tree with its current height; if the index of the new page cannot be represented with the current height, it invokes `radix_tree_extend( )` to increase the height of the tree by adding the proper number of nodes (for instance, when applied to the radix tree shown in [Figure 15-1](#) (a), `radix_tree_extend( )` would add a single node on top of it). New nodes are allocated by executing the `radix_tree_node_alloc( )` function, which tries to get a `radix_tree_node` structure from the slab allocator cache or, if this allocation fails, from the pool of preallocated structures stored in `radix_tree_preloads`.
 - b. Starting from the root (`mapping->page_tree`), it traverses the tree according to the `offset` page's index until the leaf is reached, as described in the previous section. If required, it allocates new intermediate nodes by invoking `radix_tree_node_alloc( )`.
 - c. Stores the page descriptor address in the proper slot of the last traversed node of the radix tree, and returns 0.
4. Increases the usage counter `page->_count` of the page descriptor.
5. Because the page is new, its content is invalid: the function sets the `PG_locked` flag of the page frame to protect the page against concurrent accesses from other kernel control paths.
6. Initializes `page->mapping` and `page->index` with the parameters `mapping` and `offset`.
7. Increases the counter of cached pages in the address space (`mapping->nrcpages`).
8. Releases the address space's spin lock.
9. Invokes `radix_tree_preload_end( )` to reenale kernel preemption.
10. Returns 0 (success).

15.1.3.3. Removing a page

The `remove_from_page_cache( )` function removes a page descriptor from the page cache. This is achieved in the following way:

1. Acquires the `page->mapping->tree_lock` spin lock and disables interrupts.
2. Invokes `radix_tree_delete( )` to delete the node from the tree. This function receives as its parameters the address of the tree's root (`page->mapping->page_tree`) and the index of the page to be removed and performs the following steps:
 - a. Starting from the root, it traverses the tree according to the page's index until the leaf is reached, as described in the previous section. While doing so, it builds up an array of `radix_tree_path` structures that describe the components of the path from the root to the leaf corresponding to the page to be deleted.

- b. Starts a cycle on the nodes collected in the path array, starting with the last node, which contains the pointer to the page descriptor. For each node, it sets to `NULL` the element of the slots array pointing to the next node (or to the page descriptor) and decreases the `count` field. If `count` becomes zero, it removes the node from the tree and releases the `radix_tree_node` structure to the slab allocator cache, then continues the cycle with the preceding node in the path array; otherwise, if `count` does not become zero, it continues with the next step.
 - c. Returns the pointer to the page descriptor that has been removed from the tree.
3. Sets the `page->mapping` field to `NULL`.
4. Decreases by one the `page->mapping->numpages` counter of cached pages.
5. Releases the `page->mapping->tree_lock` spin lock, enables the interrupts, and terminates.

15.1.3.4. Updating a page

The `read_cache_page( )` function ensures that the cache includes an up-to-date version of a given page. Its parameters are a pointer `mapping` to an `address_space` object, an offset value `index` that specifies the requested page, a pointer `filler` to a function that reads the page's data from disk (usually it is the function that implements the address space's `readpage` method), and a pointer `data` that is passed to the `filler` function (usually, it is `NULL`). Here is a simplified description of what the function does:

1. Invokes `find_get_page( )` to check whether the page is already in the page cache.
2. If the page is not in the page cache, it performs the following substeps:
 - a. Invokes `alloc_pages( )` to allocate a new page frame.
 - b. Invokes `add_to_page_cache( )` to insert the corresponding page descriptor into the page cache.
 - c. Invokes `lru_cache_add( )` to insert the page in the zone's inactive LRU list (see the section "[The Least Recently Used \(LRU\) Lists](#)" in [Chapter 17](#)).
3. Here the page is in the page cache. Invokes `mark_page_accessed( )` to record the fact that the page has been accessed (see the section "[The Least Recently Used \(LRU\) Lists](#)" in [Chapter 17](#)).
4. If the page is not up-to-date (`PG_uptodate` flag clear), it invokes the `filler` function to read from disk the page.
5. Returns the address of the page descriptor.

15.1.4. The Tags of the Radix Tree

As stated previously, the page cache not only allows the kernel to quickly retrieve a page containing specified data of a block device; the cache also allows the kernel to quickly retrieve pages in the cache that are in a given state.

For instance, let us suppose that the kernel must retrieve all pages in the cache that belong to a given owner and that are dirty, that is, the pages whose contents have not yet been written to disk. The `PG_dirty` flag stored in the page descriptor specifies whether a page is dirty or not; however, traversing the whole radix tree to sequentially access all the leaves—that is, the page descriptors—would be an unduly slow operation if most pages are not dirty.

Instead, to allow a quick search of dirty pages, each intermediate node in the radix tree contains a dirty tag for each child node (or leaf); this flag is set if and only if at least one of the dirty tags of the child node is set. The dirty tags of the nodes at the bottom level are usually copies of the `PG_dirty` flags of the page descriptors.

In this way, when the kernel traverses a radix tree looking for dirty pages, it can skip each subtree rooted at an intermediate node whose dirty tag is clear: it knows for sure that all page descriptors stored in the subtree are not dirty.

The same idea applies to the `PG_writeback` flag, which denotes that a page is currently being written back to disk. Thus, each node of the radix tree propagates two flags of the page descriptor: `PG_dirty` and `PG_writeback` (see the section "[Page Descriptors](#)" in [Chapter 8](#)). To store them, each node includes two arrays of 64 bits in the `tags` field. The `tags[0]` array (`PAGECACHE_TAG_DIRTY`) is the dirty tag, while the `tags[1]` (`PAGECACHE_TAG_WRITEBACK`) array is the writeback tag.

The `radix_tree_tag_set( )` function is invoked when setting the `PG_dirty` or the `PG_writeback` flag of a cached page; it acts on three parameters: the root of the radix tree, the page's index, and the type of tag to be set (`PAGECACHE_TAG_DIRTY` or `PAGECACHE_TAG_WRITEBACK`). The function starts from the root of the tree and goes down to the leaf corresponding to the given index; for each node of the path leading from the root to the leaf, the function sets the tag associated with the pointer to the next node in the path. The function then returns the address of the page descriptor. As a result, each node in the path that goes down from the root to the leaf is tagged in the appropriate way.

The `radix_tree_tag_clear( )` function is invoked when clearing the `PG_dirty` or the `PG_writeback` flag of a cached page; it acts on the same parameters as `radix_tree_tag_set( )`. The function starts from the root of the tree and goes down to the leaf, building an array of `radix_tree_path` structures describing the path. Then, the function proceeds backward from the leaf to the root: it clears the tag of the node at the bottom level, then it checks whether all tags in the node's array are now cleared; if so, the function clears the proper tag in the parent node at the upper level, checks whether all tags in that node are cleared, and so on. The function then returns the address of the page descriptor.

When a page descriptor is removed from a radix tree, the proper tags in the nodes belonging to the path from the root to the leaf must be updated. The `radix_tree_delete( )` function does this properly (even if we omitted mentioning this fact in the previous section). The `radix_tree_insert( )` function, however, doesn't update the tags, because each page descriptor inserted in the radix tree is supposed to have the `PG_dirty` and `PG_writeback` flags cleared. If necessary, the kernel may later invoke the `radix_tree_tag_set( )` function.

The `radix_tree_tagged( )` function takes advantage of the arrays of flags included in all nodes of the tree to test whether a radix tree includes at least one page in a given state. The function performs this task quite simply by executing the following code (`root` is a pointer to the `radix_tree_root` structure of the radix tree, and `tag` is the flag to be tested):

```
for (idx = 0; idx < 2; idx++) {
    if (root->rnode->tags[tag][idx])
        return 1;
}
return 0;
```

Because the tags of all nodes of the radix tree can be assumed to be properly updated, `radix_tree_tagged( )` needs only to check the tags of the node at level 1. An example of use of such function occurs when determining whether an inode contains dirty pages to be written to disk. Notice that in each iteration the function tests whether any of the 32 flags stored in an `unsigned long` is set.

The `find_get_pages_tag( )` function is similar to `find_get_pages( )` except that it returns only pages that are tagged with the `tag` parameter. As we'll see in the section "[Writing Dirty Pages to Disk](#)," this function is crucial to quickly identify all the dirty pages of an inode.

15.2. Storing Blocks in the Page Cache

We have seen in the section "[Block Devices Handling](#)" in [Chapter 14](#) that the VFS, the mapping layer, and the various filesystems group the disk data in logical units called "blocks."

In old versions of the Linux kernel, there were two different main disk caches: the page cache, which stored whole pages of disk data resulting from accesses to the contents of the disk files, and the buffer cache , which was used to keep in memory the contents of the blocks accessed by the VFS to manage the disk-based filesystems.

Starting from stable version 2.4.10, the buffer cache does not really exist anymore. In fact, for reasons of efficiency, block buffers are no longer allocated individually; instead, they are stored in dedicated pages called "buffer pages ," which are kept in the page cache.

Formally, a buffer page is a page of data associated with additional descriptors called "buffer heads ," whose main purpose is to quickly locate the disk address of each individual block in the page. In fact, the chunks of data stored in a page belonging to the page cache are not necessarily adjacent on disk.

15.2.1. Block Buffers and Buffer Heads

Each block buffer has a buffer head descriptor of type `buffer_head`. This descriptor contains all the information needed by the kernel to know how to handle the block; thus, before operating on each block, the kernel checks its buffer head. The fields of a buffer head are listed in [Table 15-4](#).

Table 15-4. The fields of a buffer head

Type	Field	Description
unsigned long	b_state	Buffer status flags
struct buffer_head *	b_this_page	Pointer to the next element in the buffer page's list
		Pointer to the descriptor of the buffer page holding this block
struct page *	b_page	
atomic_t	b_count	Block usage counter

u32	b_size	Block size
		Block number
		relative to the
sector_t	b_blocknr	block device
		(logical block
		number)
char *	b_data	Position of the
		block inside the
		buffer page
struct block_device *	b_bdev	Pointer to block
		device descriptor
bh_end_io_t *	b_end_io	I/O completion
		method
void *	b_private	Pointer to data for
		the I/O completion
		method
struct list_head	b_assoc_buffers	Pointers for the list
		of indirect blocks
		associated with an
		inode (see the
		section " The
		address_space
		Object " earlier in
		this chapter)

Two fields of the buffer head encode the disk address of the block: the `b_bdev` field identifies the block device usually, a disk or a partition that contains the block (see the section "[Block Devices](#)" in [Chapter 14](#)), while the `b_blocknr` field stores the logical block number, that is, the index of the block inside its disk or partition.

The `b_data` field specifies the position of the block buffer inside the buffer page. Actually, the encoding of this position depends on whether the page is in high memory or not. If the page is in high memory, the `b_data` field contains the offset of the block buffer with respect to the beginning of the page; otherwise, `b_data` contains the linear address of the block buffer.

The `b_state` field may store several flags. Some of them are of general use and are listed in [Table 15-5](#). Each filesystem may also define its own private buffer head flags.

Table 15-5. The buffer head's general flags

Flag

Description

`BH_Uptodate`

Set if the buffer contains valid data

`BH_Dirty`

Set if the buffer is dirty that is, it contains data that must be written to the block device

`BH_Lock`

Set if the buffer is locked, which usually happens when the buffer is involved in a disk transfer

`BH_Req`

Set if data transfer for initializing the buffer has already been requested

`BH_Mapped`

Set if the buffer is mapped to disk that is, if the `b_bdev` and `b_blocknr` fields of the corresponding buffer head are significant

`BH_New`

Set if the corresponding block has just been allocated and has never been accessed

`BH_Async_Read`

Set if the buffer is being read asynchronously

`BH_Async_Write`

Set if the buffer is being written asynchronously

`BH_Delay`

Set if the buffer is not yet allocated on disk

`BH_Boundary`

Set if the block to be submitted after this one will not be adjacent to this one

`BH_Write_EIO`

Set if there was an I/O error when writing this block

`BH_Ordered`

Set if the block should be written strictly after the blocks submitted before it (used by journaling filesystems)

`BH_Eopnotsupp`

Set if the block device driver does not support the operation requested

15.2.2. Managing the Buffer Heads

The buffer heads have their own slab allocator cache, whose `kmem_cache_s` descriptor is stored in the `bh_cache` variable. The `alloc_buffer_head( )` and `free_buffer_head( )` functions are used to get and release a buffer head, respectively.

The `b_count` field of the buffer head is a usage counter for the corresponding block buffer. The counter is increased right before each operation on the block buffer and decreased right after. The block buffers kept in the page cache are examined both periodically and when free memory becomes scarce, and only the block buffers having null usage counters may be reclaimed (see [Chapter 17](#)).

When a kernel control path wishes to access a block buffer, it should first increase the usage counter. The function that locates a block inside the page cache (`__getblk()`; see the section "[Searching Blocks in the Page Cache](#)" later in this chapter) does this automatically, hence the higher-level functions do not usually increase the block buffer's usage counter.

When a kernel control path stops accessing a block buffer, it should invoke either `__brelse()` or `__bforget()` to decrease the corresponding usage counter. The difference between these two functions is that `__bforget()` also removes the block from any list of indirect blocks (`b_assoc_buffers` buffer head field; see the previous section "[Block Buffers and Buffer Heads](#)") and marks the buffer as clean, thus forcing the kernel to forget any change in the buffer that has yet to be written on disk.

15.2.3. Buffer Pages

Whenever the kernel must individually address a block, it refers to the buffer page that holds the block buffer and checks the corresponding buffer head.

Here are two common cases in which the kernel creates buffer pages:

- When reading or writing pages of a file that are not stored in contiguous disk blocks. This happens either because the filesystem has allocated noncontiguous blocks to the file, or because the file contains "holes" (see the section "[File Holes](#)" in [Chapter 18](#)).
- When accessing a single disk block (for instance, when reading a superblock or an inode block).

In the first case, the buffer page's descriptor is inserted in the radix tree of a regular file. The buffer heads are preserved because they store precious information: the block device and the logical block number that specify the position of the data in the disk. We will see how the kernel makes use of this type of buffer page in [Chapter 16](#).

In the second case, the buffer page's descriptor is inserted in the radix tree rooted at the `address_space` object of the inode in the `bdev` special filesystem associated with the block device (see the section "[The address_space Object](#)" earlier in this chapter). This kind of buffer pages must satisfy a strong constraint: all the block buffers must refer to adjacent blocks of the underlying block device.

An instance of where this is useful is when the VFS wants to read the 1,024-byte inode block containing the inode of a given file. Instead of allocating a single buffer, the kernel must allocate a whole page storing four buffers; these buffers will contain the data of a group of four adjacent blocks on the block device, including the requested inode block.

In this chapter we will focus our attention on the second type of buffer pages, the so-called block device buffer pages (sometimes shortened to `blockdev's` pages).

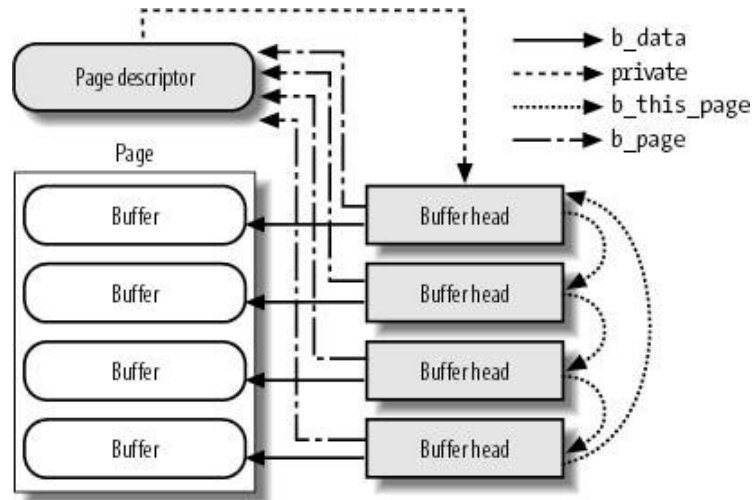
All the block buffers within a single buffer page must have the same size; hence, on the 80 x 86 architecture, a buffer page can include from one to eight buffers, depending on the block size.

When a page acts as a buffer page, all buffer heads associated with its block buffers are collected in a singly linked circular list. The `private` field of the descriptor of the buffer page points to the buffer head of the

first block in the page;[*] every buffer head stores in the `b_this_page` field a pointer to the next buffer head in the list. Moreover, every buffer head stores the address of the buffer page's descriptor in the `b_page` field. Figure 15-2 shows a buffer page containing four block buffers and the corresponding buffer heads.

[*] Because the `private` field contains valid data, the `PG_private` flag of the page is also set; hence, if the page contains disk data and the `PG_private` flag is set, then the page is a buffer page. Notice, however, that other kernel components not related to the block I/O subsystem use the `private` and `PG_private` fields for other purposes.

Figure 15-2. A buffer page including four buffers and their buffer heads



15.2.4. Allocating Block Device Buffer Pages

The kernel allocates a new block device buffer page when it discovers that the page cache does not include a page containing the buffer for a given block (see the section "[Searching Blocks in the Page Cache](#)" later in this chapter). In particular, the lookup operation for the block might fail for the following reasons:

1. The radix tree of the block device does not include a page containing the data of the block: in this case a new page descriptor must be added to the radix tree.
2. The radix tree of the block device includes a page containing the data of the block, but this page is not a buffer page: in this case new buffer heads must be allocated and linked to the page, thus transforming it into a block device buffer page.
3. The radix tree of the block device includes a buffer page containing the data of the block, but the page has been split in blocks of size different from the size of the requested block: in this case the old buffer heads must be released, and a new set of buffer heads must be allocated and linked to the page.

In order to add a block device buffer page to the page cache, the kernel invokes the `grow_buffers( )` function, which receives three parameters that identify the block:

- The address `bdev` of the `block_device` descriptor
- The logical block number `block` the position of the block inside the block device
- The block size `size`

The function essentially performs the following actions:

1. Computes the offset `index` of the page of data within the block device that includes the requested block.
2. Invokes `grow_dev_page( )` to create a new block device buffer page, if necessary. In turn, this function performs the following substeps:
 - a. Invokes `find_or_create_page( )`, passing to it the `address_space` object of the block device (`bdev->bd_inode->i_mapping`), the page offset `index`, and the `GFP_NOFS` flag. As described in the earlier section "[Page Cache Handling Functions](#)," `find_or_create_page( )` looks for the page in the page cache and, if necessary, inserts a new page in the cache.
 - b. Now the required page is in the page cache, and the function has the address of its descriptor. The function checks its `PG_private` flag; if it is `NULL`, the page is not yet a buffer page (it has no associated buffer heads): it jumps to step 2e.
 - c. The page is already a buffer page. Gets from the `private` field of its descriptor the address `bh` of the first buffer head, and checks whether the block size `bh->size` is equal to the size of the requested block; if so, the page found in the page cache is a valid buffer page: it jumps to step 2g.
 - d. The page has blocks of the wrong size: it invokes `try_to_free_buffers( )` (see the next section) to release the previous buffer heads of the buffer page.
 - e. Invokes the `alloc_page_buffers( )` function to allocate the buffer heads for the blocks of the requested size within the page and insert them into the singly linked circular list implemented by the `b_this_page` fields. Moreover, the function initializes the `b_page` fields of the buffer heads with the address of the page descriptor, and the `b_data` fields with the offset or linear address of the block buffer inside the page.
 - f. Stores the address of the first buffer head in the `private` field, sets the `PG_private` field, and increases the usage counter of the page (the block buffers inside the page counts as a page user).
 - g. Invokes the `init_page_buffers( )` function to initialize the `b_bdev`, `b_blocknr`, and `b_bstate` fields of the buffer heads linked to the page. All blocks are adjacent on disk, hence the logical block numbers are consecutive and can be easily derived from `block`.
 - h. Returns the page descriptor address.
3. Unlocks the page (the page was locked by `find_or_create_page( )`).
4. Decreases the page's usage counter (again, the counter was increased by `find_or_create_page( )`).
5. Returns 1 (success).

15.2.5. Releasing Block Device Buffer Pages

As we will see in [Chapter 17](#), block device buffer pages are released when the kernel tries to get additional free memory. Clearly a buffer page cannot be freed if it contains dirty or locked buffers. To release buffer pages, the kernel invokes the `try_to_release_page( )` function, which receives the address `page` of a page descriptor and performs the following actions:^[*]

[*] The `try_to_release_page( )` function can also be invoked on buffer pages owned by regular files.

1. If the `PG_writeback` flag of the page is set, it returns 0 (no release is possible because the page is being written back to disk).
2. If defined, it invokes the `releasepage` method of the block device's `address_space` object.

(The method is usually not defined for block devices.)

3. Invokes the `try_to_free_buffers( )` function, and returns its error code.

In turn, the `try_to_free_buffers( )` function scans the buffer heads linked to the buffer page; it performs essentially the following actions:

1. Checks the flags of all the buffer heads of buffers included in the page. If some buffer head has the `BH_Dirty` or `BH_Locked` flag set, the function terminates by returning 0 (failure): it is not possible to release the buffers.
2. If a buffer head is inserted in a list of indirect buffers (see the section "[Block Buffers and Buffer Heads](#)" earlier in this chapter), the function removes it from the list.
3. Clears the `PG_private` flag of the page descriptor, sets the `private` field to `NULL`, and decreases the page's usage counter.
4. Clears the `PG_dirty` flag of the page.
5. Invokes repeatedly `free_buffer_head( )` on the buffer heads of the page to free all of them.
6. Returns 1 (success).

15.2.6. Searching Blocks in the Page Cache

When the kernel needs to read or write a single block of a physical device (for instance, a superblock), it must check whether the required block buffer is already included in the page cache. Searching the page cache for a given block bufferspecified by the address `bdev` of a block device descriptor and by a logical block number `nr` is a three stage process:

1. Get a pointer to the `address_space` object of the block device containing the block (`bdev->bd_inode->i_mapping`).
2. Get the block size of the device (`bdev->bd_block_size`), and compute the index of the page that contains the block. This is always a bit shift operation on the logical block number. For instance, if the block size is 1,024 bytes, each buffer page contains four block buffers, thus the page's index is `nr/4`.
3. Searches for the buffer page in the radix tree of the block device. After obtaining the page descriptor, the kernel has access to the buffer heads that describe the status of the block buffers inside the page.

Details are slightly more complicated than this, however. In order to enhance system performance, the kernel manages a `bh_lrus` array of small disk caches, one for each CPU, called the Least Recently Used (LRU) block cache. Each disk cache contains eight pointers to buffer heads that have been recently accessed by a given CPU. The elements in each CPU array are sorted so that the pointer to the most recently used buffer head has index 0. The same buffer head might appear on several CPU arrays (but never twice in the same CPU array); for each occurrence of a buffer head in the LRU block cache, the buffer head's `b_count` usage counter is increased by one.

15.2.6.1. The `__find_get_block( )` function

The `__find_get_block( )` function receives as its parameters the address `bdev` of a `block_device` descriptor, the block number `block`, and the block size `size`, and returns the address of the buffer head associated with the block buffer inside the page cache, or `NULL` if no such block buffer exists. The function performs essentially the following actions:

1. Checks whether the LRU block cache array of the executing CPU includes a buffer head whose `b_bdev`, `b_blocknr`, and `b_size` fields are equal to `bdev`, `block`, and `size`, respectively.

2. If the buffer head is in the LRU block cache, it reshuffles the elements in the array so as to put the pointer to the just discovered buffer head in the first position (index 0), increases its `b_count` field, and jumps to step 8.
3. Here the buffer head is not in the LRU block cache: it derives from the block number and the block size the page index relative to the block device as:

```
index = block >> (PAGE_SHIFT - bdev->bd_inode->i_blkbits);
```

4. Invokes `find_get_page( )` to locate, in the page cache, the descriptor of the buffer page containing the required block buffer. The function passes as parameters a pointer to the `address_space` object of the block device (`bdev->bd_inode->i_mapping`) and the page index to locate in the page cache the descriptor of the buffer page containing the required block buffer. If there is no such page in the cache, returns `NULL` (failure).
5. At this point, the function has the address of a descriptor for the buffer page: it scans the list of buffer heads linked to the buffer page, looking for the block having logical block number equal to `block`.
6. Decreases the `count` field of the page descriptor (it was increased by `find_get_page( )`).
7. Moves all elements in the LRU block cache one position down, and inserts the pointer to the buffer head of the requested block in the first position. If a buffer head has been dropped out of the LRU block cache, it decreases its `b_count` usage counter.
8. Invokes `mark_page_accessed( )` to move the buffer page in the proper LRU list, if necessary (see the section "[The Least Recently Used \(LRU\) Lists](#)" in [Chapter 17](#)).
9. Returns the buffer head pointer.

15.2.6.2. The `__getblk( )` function

The `__getblk( )` function receives the same parameters as `__find_get_block( )`, namely the address `bdev` of a `block_device` descriptor, the block number `block`, and the block size `size`, and returns the address of a buffer head associated with the buffer. The function never fails: even if the block does not exist at all, the `__getblk( )` obligingly allocates a block device buffer page and returns a pointer to the buffer head that should describe the block. Notice that the block buffer returned by `__getblk( )` does not necessarily contain valid data: the `BH_Uptodate` flag of the buffer head might be cleared.

The `__getblk( )` function essentially performs the following steps:

1. Invokes `__find_get_block( )` to check whether the block is already in the page cache. If the block is found, the function returns the address of its buffer head.
2. Otherwise, it invokes `grow_buffers( )` to allocate a new buffer page for the requested block (see the section "[Allocating Block Device Buffer Pages](#)" earlier in this chapter).
3. If `grow_buffers( )` fails in allocating such a page, `__getblk( )` tries to reclaim some memory by invoking `free_more_memory( )` (see [Chapter 17](#)).
4. Jumps back to step 1.

15.2.6.3. The `__bread( )` function

The `__bread( )` function receives the same parameters as `__getblk( )`, namely the address `bdev` of a `block_device` descriptor, the block number `block`, and the block size `size`, and returns the address of a buffer head associated with the buffer. Contrary to `__getblk( )`, the function reads the block from disk, if necessary, before returning the buffer head. The `__bread( )` function performs the following steps:

1. Invokes `__getblk( )` to find in the page cache the buffer page associated with the required block and to get a pointer to the corresponding buffer head.
2. If the block is already in the page cache and the buffer contains valid data (flag `BH_Uptodate` set), it returns the address of the buffer head.
3. Otherwise, it increases the usage counter of the buffer head.
4. Sets the `b_end_io` field to the address of `end_buffer_read_sync( )` (see the next section).
5. Invokes `submit_bh( )` to transmit the buffer head to the generic block layer (see next section).
6. Invokes `wait_on_buffer( )` to put the current process in a wait queue until the read I/O operation is completed, that is, until the `BH_Lock` flag of the buffer head is cleared.
7. Returns the address of the buffer head.

15.2.7. Submitting Buffer Heads to the Generic Block Layer

A couple of functions, `submit_bh( )` and `ll_rw_block( )`, allow the kernel to start an I/O data transfer on one or more buffers described by their buffer heads.

15.2.7.1. The `submit_bh( )` function

To transmit a single buffer head to the generic block layer, and thus to require the transfer of a single block of data, the kernel makes use of the `submit_bh( )` function. Its parameters are the direction of data transfer (essentially `READ` or `WRITE`) and a pointer `bh` to the buffer head describing the block buffer.

The `submit_bh( )` function assumes that the buffer head is fully initialized; in particular, the `b_bdev`, `b_blocknr`, and `b_size` fields must be properly set to identify the block on disk containing the requested data. If the block buffer belongs to a block device buffer page, the initialization of the buffer head is done by `__find_get_block( )`, as described in the previous section. However, as we will see in the next chapter, `submit_bh( )` can also be invoked on blocks belonging to buffer pages owned by regular files.

The `submit_bh( )` function is little else than a glue function that creates a bio request from the contents of the buffer head and then invokes `generic_make_request( )` (see the section "[Submitting a Request](#)" in [Chapter 14](#)). The main steps performed by it are the following:

1. Sets the `BH_Req` flag of the buffer head to record that the block has been submitted at least one time; moreover, if the direction of the data transfer is `WRITE`, clears the `BH_Write_EIO` flag.
2. Invokes `bio_alloc( )` to allocate a new bio descriptor (see the section "[The Bio Structure](#)" in [Chapter 14](#)).
3. Initializes the fields of the bio descriptor according to the contents of the buffer head:
 - a. Sets the `bi_sector` field to the number of the first sector in the block (`bh->b_blocknr * bh->b_size / 512`);
 - b. Sets the `bi_bdev` field with the address of the block device descriptor (`bh->b_bdev`);
 - c. Sets the `bi_size` field with the block size (`bh->b_size`);
 - d. Initializes the first element of the `bi_io_vec` array so that the segment corresponds to the block buffer: `bi_io_vec[0].bv_page` is set to `bh->b_page`, `bi_io_vec[0].bv_len` is set to `bh->b_size`, and `bi_io_vec[0].bv_offset` is set to the offset of the block buffer in the page as specified by `bh->b_data`;
 - e. Sets `bi_vcnt` to 1 (just one segment on the bio), and `bi_idx` to 0 (the current segment to be transferred);
 - f. Sets the `bi_end_io` field to the address of `end_bio_bh_io_sync( )`, and sets the `bi_private` field to the address of the buffer head; the function will be invoked when the data transfer terminates (see below).

4. Increases the reference counter of the bio (it becomes equal to 2).
5. Invokes `submit_bio( )`, which sets the `bi_rw` flag with the direction of the data transfer, updates the `page_states` per-CPU variable to keep track of the number of sectors read and written, and invokes the `generic_make_request( )` function on the bio descriptor.
6. Decreases the usage counter of the bio; the bio descriptor is not freed, because it is now inserted in a queue of the I/O scheduler.
7. Returns 0 (success).

When the I/O data transfer on the bio terminates, the kernel executes the `bi_end_io` method, in this particular case the `end_bio_bh_io_sync( )` function. The latter function essentially gets the address of the buffer head from the `bi_private` field of the bio, then invokes the `b_end_io` method of the buffer head it was properly set before invoking `submit_bh( )` and finally invokes `bio_put( )` to destroy the bio structure.

15.2.7.2. The `ll_rw_block( )` function

Sometimes the kernel must trigger the data transfer of several data blocks at once, which are not necessarily physically adjacent. The `ll_rw_block( )` function receives as its parameters the direction of data transfer (essentially READ or WRITE), the number of blocks to be transferred, and an array of pointers to buffer heads describing the corresponding block buffers. The function iterates over all buffer heads; for each of them, it executes the following actions:

1. Tests and sets the `BH_Lock` flag of the buffer head; if the buffer was already locked, the data transfer has been activated by another kernel control path, so just skips the buffer by jumping to step 9.
2. Increases by one the usage counter `b_count` of the buffer head.
3. If the data transfer direction is WRITE, it sets the `b_end_io` method of the buffer head to point to the address of the `end_buffer_write_sync( )` function; otherwise, it sets the `b_end_io` method to point to the address of the `end_buffer_read_sync( )` function.
4. If the data transfer direction is WRITE, it tests and clears the `BH_Dirty` flag of the buffer head. If the flag was not set, there is no need to write the block on disk, so it jumps to step 7.
5. If the data transfer direction is READ or READA (read-ahead), it checks whether the `BH_Uptodate` flag of the buffer head is set; if so, there is no need to read the block from disk, so it jumps to step 7.
6. Here the block has to be read or written: it invokes the `submit_bh( )` function to pass the buffer head to the generic block layer, then jumps to step 9.
7. Unlocks the buffer head by clearing the `BH_Lock` flag, and awakens every process that was waiting for the block being unlocked.
8. Decreases the `b_count` field of the buffer head.
9. If there are other buffer heads in the array to be processed, it selects the next one and jumps back to step 1; otherwise, it terminates.

Notice that if the `ll_rw_block( )` function passes a buffer head to the generic block layer, it leaves the buffer locked and its reference counter increased, so that the buffer cannot be accessed and cannot be freed until the data transfer completes. The kernel executes the `b_end_io` completion method of the buffer head when the data transfer for the block terminates. Assuming that there was no I/O error, the `end_buffer_write_sync( )` and `end_buffer_read_sync( )` functions simply set the `BH_Uptodate` field of the buffer head, unlock the buffer, and decrease its usage counter.

15.3. Writing Dirty Pages to Disk

As we have seen, the kernel keeps filling the page cache with pages containing data of block devices. Whenever a process modifies some data, the corresponding page is marked as dirty that is, its `PG_dirty` flag is set.

Unix systems allow the deferred writes of dirty pages into block devices, because this noticeably improves system performance. Several write operations on a page in cache could be satisfied by just one slow physical update of the corresponding disk sectors. Moreover, write operations are less critical than read operations, because a process is usually not suspended due to delayed writings, while it is most often suspended because of delayed reads. Thanks to deferred writes, each physical block device will service, on the average, many more read requests than write ones.

A dirty page might stay in main memory until the last possible moment that is, until system shutdown. However, pushing the delayed-write strategy to its limits has two major drawbacks:

- If a hardware or power supply failure occurs, the contents of RAM can no longer be retrieved, so many file updates that were made since the system was booted are lost.
- The size of the page cache, and hence of the RAM required to contain it, would have to be huge at least as big as the size of the accessed block devices.

Therefore, dirty pages are flushed (written) to disk under the following conditions:

- The page cache gets too full and more pages are needed, or the number of dirty pages becomes too large.
- Too much time has elapsed since a page has stayed dirty.
- A process requests all pending changes of a block device or of a particular file to be flushed; it does this by invoking a `sync( )`, `fsync( )`, or `fdatasync( )` system call (see the section "[The sync\(\), fsync\(\), and fdatasync\(\) System Calls](#)" later in this chapter).

Buffer pages introduce a further complication. The buffer heads associated with each buffer page allow the kernel to keep track of the status of each individual block buffer. The `PG_dirty` flag of the buffer page should be set if at least

one of the associated buffer heads has the `BH_Dirty` flag set. When the kernel selects a dirty buffer page for flushing, it scans the associated buffer heads and effectively writes to disk only the contents of the dirty blocks. As soon as the kernel flushes all dirty blocks in a buffer page to disk, it clears the `PG_dirty` flag of the page.

15.3.1. The `pdflush` Kernel Threads

Earlier versions of Linux used a kernel thread called `bdfush` to systematically scan the page cache looking for dirty pages to flush, and they used a second kernel thread called `kupdate` to ensure that no page remains dirty for too long. Linux 2.6 has replaced both of them with a group of general purpose kernel threads called `pdflush`.

These kernel threads have a flexible structure. They act on two parameters: a pointer to a function to be executed by the thread and a parameter for the function. The number of *pdflush* kernel threads in the system is dynamically adjusted: new threads are created when they are too few and existing threads are killed when they are too many. Because the functions executed by these kernel threads can block, creating several `pdflush` kernel threads instead of a single one, leads to better system performance.

Births and deaths are governed by the following rules:

- There must be at least two `pdflush` kernel threads and at most eight.
- If there were no idle `pdflush` during the last second, a new `pdflush` should be created.
- If more than one second elapsed since the last `pdflush` became idle, a `pdflush` should be removed.

Each `pdflush` kernel thread has a `pdflush_work` descriptor (see [Table 15-6](#)). The descriptors of idle `pdflush` kernel threads are collected in the `pdflush_list` list; the `pdflush_lock` spin lock protects that list from concurrent accesses in multiprocessor systems. The `nr_pdflush_threads` variable^[*] stores the total number of `pdflush` kernel threads (idle and busy). Finally, the `last_empty_jifs` variable stores the last time (in jiffies) since the `pdflush_list` list of `pdflush` threads became empty.

[*] The value of this variable can be read from the `/proc/sys/vm/nr_pdflush_threads` file.

Table 15-6. The fields of the pdflush_work descriptor

Type	Field	Description
<code>struct task_struct *</code>	<code>who</code>	Pointer to kernel thread descriptor
<code>void(*) (unsigned long)</code>	<code>fn</code>	Callback function to be executed by the kernel thread
<code>unsigned long</code>	<code>arg0</code>	Argument to callback function
<code>struct list_head</code>	<code>list</code>	Links for the <code>pdflush_list</code> list
<code>unsigned long</code>	<code>when_i_went_to_sleep</code>	Time in jiffies when kernel thread became available

Each `pdflush` kernel thread executes the `_ _pdflush( )` function, which essentially loops in an endless cycle until the kernel thread dies. Let's suppose that the `pdflush` kernel thread is idle; then, the process is sleeping in `TASK_INTERRUPTIBLE` state. As soon as the kernel thread is woken up, `_ _pdflush( )` accesses its `pdflush_work` descriptor and executes the callback function stored in the `fn` field, passing to it the argument stored in the `arg0` field. When the callback function terminates, `_ _pdflush( )` checks the value of the `last_empty_jifs` variable: if there was no idle `pdflush` kernel thread for more than one second and if there are less than eight `pdflush` kernel threads, `_ _pdflush( )` starts another kernel thread. Otherwise, if the last entry in the `pdflush_list` list is idle for more than one second, and there are more than two `pdflush` kernel threads, `_ _pdflush( )` terminates: as explained in the section "[Kernel Threads](#)" in [Chapter 3](#), the corresponding kernel thread executes the `_exit( )` system call and it is thus destroyed. Otherwise, `_ _pdflush( )` reinserts the `pdflush_work` descriptor of the kernel thread in the `pdflush_list` list and puts the kernel thread to sleep.

The `pdflush_operation( )` function is used to activate an idle `pdflush` kernel thread. This function acts on two parameters: a pointer `fn` to the function that must be executed and an argument `arg0`; it performs the following steps:

1. Extracts from the `pdflush_list` list a pointer `pdf` to the `pdflush_work` descriptor of an idle `pdflush` kernel thread. If the list is empty, it returns `-1`. If the list contained just one element, it sets the value of the `last_empty_jifs` variable to `jiffies`.
2. Stores in `pdf->fn` and in `pdf->arg0` the parameters `fn` and `arg0`.
3. Invokes `wake_up_process( )` to wake up the idle `pdflush` kernel thread, that is, `pdf->who`.

What kinds of jobs are delegated to the `pdflush` kernel threads? There are a few of them, all related to flushing of dirty data. In particular, `pdflush` usually executes one of the following callback functions:

- `background_writeout( )`: systematically walks the page cache looking for dirty pages to be flushed (see the next section "[Looking for Dirty Pages To Be Flushed](#)").
- `wb_kupdate( )`: checks that no page in the page cache remains dirty for too long (see the section "[Retrieving Old Dirty Pages](#)" later in this chapter).

15.3.2. Looking for Dirty Pages To Be Flushed

Every radix tree could include dirty pages to be flushed. Retrieving all of them thus involves an exhaustive search among all `address_space` objects associated with inodes having an image on disk. Because the page cache might include a large number of pages, scanning the whole cache in a single run might keep the CPU and the disks busy for a long time. Therefore, Linux adopts a sophisticated mechanism that splits the page cache scanning in several runs of execution.

The `wakeup_bdflush( )` function receives as argument the number of dirty pages in the page cache that should be flushed; the value zero means that all dirty pages in the cache should be written back to disk. The function invokes `pdflush_operation( )` to wake up a `pdflush` kernel thread (see the previous section) and delegate to it the execution of the `background_writeout( )` callback function. The latter function effectively retrieves the specified number of dirty pages from the page cache and writes them back to disk.

The `wakeup_bdflush( )` function is executed when either memory is scarce or a user makes an explicit request for a flush operation. In particular, the function is invoked when:

- The User Mode process issues a `sync( )` system call (see the section "[The sync\(\), fsync\(\), and fdatasync\(\) System Calls](#)" later in this chapter).
- The `grow_buffers( )` function fails to allocate a new buffer page (see the earlier section "[Allocating Block Device Buffer Pages](#)").
- The page frame reclaiming algorithm invokes `free_more_memory( )` or `try_to_free_pages( )` (see [Chapter 17](#)).
- The `mempool_alloc( )` function fails to allocate a new memory pool element (see the section "[Memory Pools](#)" in [Chapter 8](#)).

Moreover, a `pdflush` kernel thread executing the `background_writeout( )` callback function is woken up by every process that modifies the contents of pages in the page cache and causes the fraction of dirty pages to rise above some dirty background threshold. The background threshold is typically set to 10% of all pages in the system, but its value can be adjusted by writing in the `/proc/sys/vm/dirty_background_ratio` file.

The `background_writeout( )` function relies on a `writeback_control` structure, which acts as a two-way communication device: on one hand, it tells an auxiliary function called `writeback_inodes( )` what to do; on the other hand, it stores some statistics about the number of pages written to disk. The most important fields of this structure are the following:

`sync_mode`

Specifies the synchronization mode: `WB_SYNC_ALL` means that if a locked inode is encountered, it must be waited upon and not just skipped over; `WB_SYNC_HOLD` means that locked inodes are put in a list for later consideration; and `WB_SYNC_NONE` means that locked inodes are simply skipped.

`bdi`

If not `NULL`, it points to a `backing_dev_info` structure; in this case, only dirty pages belonging to the underlying block device will be flushed.

`older_than_this`

If not null, it means that inodes younger than the specified value should be skipped.

`nr_to_write`

Number of dirty pages yet to be written in this run of execution.

nonblocking

If this flag is set, the process cannot be blocked.

The `background_writeout ( )` function acts on a single parameter: `nr_pages`, the minimum number of pages that should be flushed to disk. It essentially executes the following steps:

1. Reads from the `page_state` per-CPU variable the number of pages and dirty pages currently stored in the page cache. If the fraction of dirty pages is below a given threshold and at least `nr_pages` have been flushed to disk, the function terminates. The value of this threshold is typically set to about 40% of the number of pages in the system; it could be adjusted by writing into the `/proc/sys/vm/dirty_ratio` file.
2. Invokes `writeback_inodes ( )` to try to write 1,024 dirty pages (see below).
3. Checks the number of pages effectively written and decreases the number of pages yet to be written.
4. If less than 1,024 pages have been written or if pages have been skipped, probably the request queue of the block device is congested: the function puts the current process to sleep in a special wait queue for 100 milliseconds or until the queue becomes uncongested.
5. Goes back to step 1.

The `writeback_inodes ( )` function acts on a single parameter, namely a pointer `wbc` to a `writeback_control` descriptor. The `nr_to_write` field of this descriptor contains the number of pages to be flushed to disk. When the function returns, the same field contains the number of pages remaining to be flushed; if everything went smoothly, this field will be set to 0.

Let us suppose that `writeback_inodes ( )` is called with the `wbc->bdi` and `wbc->older_than_this` pointers set to `NULL`, the `WB_SYNC_NONE` synchronization mode, and the `wbc->nonblocking` flag set; these are the values set by `background_writeout ( )`. The function scans the list of superblocks rooted at the `super_blocks` variable (see the section "[Superblock Objects](#)" in [Chapter 12](#)). The scanning ends when either the whole list has been traversed, or the target number of pages to be flushed has been reached. For each superblock `sb`, the function executes the following steps:

1. Checks whether the `sb->s_dirty` or `sb->s_io` lists are empty: the first list collects the dirty inodes of the superblock, while the second list collects the inodes waiting to be transferred to disk (see below). If both lists are empty, the inodes on this filesystem have no dirty pages, so the function considers the next superblock in the list.
2. Here the superblock has dirty inodes. Invokes `sync_sb_inodes ( )` on the `sb` superblock. This function:
 - a. Puts all the inodes of `sb->s_dirty` into the list pointed to by `sb->s_io` and clears the list of dirty inodes.
 - b. Gets the next `inode` pointer from `sb->s_io`. If this list is empty, it returns.
 - c. If the inode was dirtied after `sync_sb_inodes ( )` started, it skips the inode's dirty pages and returns. Notice that some dirty inodes might remain in the `sb->s_io` list.
 - d. If the current process is a `pdflush` kernel thread, it checks whether another `pdflush` kernel thread running on another CPU is already trying to flush dirty pages for files belonging to this block device. This can be done by an atomic test and set operation on the `BDI_pdflush` flag of the inode's `backing_dev_info`. Essentially, it is pointless to have more than one `pdflush` kernel thread on the same request queue (see the section "[The pdflush Kernel Threads](#)" earlier in this chapter).
 - e. Increases by one the inode's usage counter.
 - f. Invokes `__writeback_single_inode ( )` to write back the dirty buffers associated with the selected inode:
 1. If the inode is locked, it moves `inode` into the list of dirty inodes (`inode->i_sb->s_dirty`) and returns 0. (Since we are assuming that the

- wbc->sync_mode field is not WB_SYNC_ALL, the function does not block waiting for the inode to unlock.)
2. Uses the `writepages` method of the inode's address space, or the `mpage_writepages ( )` function if no such method exists, to write up to `wbc->nr_to_write` dirty pages. This function uses the `find_get_pages_tag ( )` function to retrieve quickly all dirty pages in the inode's address space (see the section "[The Tags of the Radix Tree](#)" earlier in this chapter). Details will be given in the next chapter.
 3. If the inode is dirty, it uses the superblock's `write_inode` method to write the inode to disk. The functions that implement this method usually rely on `submit_bh ( )` to transfer a single block of data (see the section "[Submitting Buffer Heads to the Generic Block Layer](#)" earlier in this chapter).
 4. Checks the status of the inode; accordingly, moves the inode back into the `sb->s_dirty` list if some page of the inode is still dirty, or in the `inode_unused` list if the inode's reference counter is zero, or in the `inode_in_use` list otherwise (see the section "[Inode Objects](#)" in [Chapter 12](#)).
 5. Returns the error code of the function invoked in step 2f2.
- g. Back into the `sync_sb_inodes ( )` function. If the current process is the `pdflush` kernel thread, it clears the `BDI_pdflush` flag set in step 2d.
 - h. If some pages were skipped in the inode just processed, then the inode includes locked buffers: moves all inodes remaining in the `sb->s_io` list back into the `sb->s_dirty` list: they will be reconsidered at a later time.
 - i. Decreases by one the usage counter of the inode.
 - j. If `wbc->nr_to_write` is greater than 0, goes back to step 2b to look for other dirty inodes of the same superblock. Otherwise, the `sync_sb_inodes ( )` function terminates.
3. Back into the `writeback_inodes ( )` function. If `wbc->nr_to_write` is greater than zero, it jumps to step 1 and continues with the next superblock in the global list. Otherwise, it returns.

15.3.3. Retrieving Old Dirty Pages

As stated earlier, the kernel tries to avoid the risk of starvation that occurs when some pages are not flushed for a long period of time. Hence, if a page remains dirty for a predefined amount of time, the kernel explicitly starts an I/O data transfer that writes its contents to disk.

The job of retrieving old dirty pages is delegated to a `pdflush` kernel thread that is periodically woken up. During the kernel initialization, the `page_writeback_init ( )` function sets up the `wb_timer` dynamic timer so that it decays after `dirty_writeback_centisecs` hundreds of a second (usually 500, but this value can be adjusted by writing in the `/proc/sys/vm/dirty_writeback_centisecs` file). The timer function, which is called `wb_timer_fn ( )`, essentially invokes the `pdflush_operation ( )` function passing to it the address of the `wb_kupdate ( )` callback function.

The `wb_kupdate ( )` function walks the page cache looking for "old" dirty inodes; it executes the following steps:

1. Invokes the `sync_supers ( )` function to write the dirty superblocks to disk (see the next section). Although not strictly related to the flushing of the pages in the page cache, this invocation ensures that no superblock remains dirty for more than, usually, five seconds.
2. Stores in the `older_than_this` field of a `writeback_control` descriptor a pointer to a value in jiffies corresponding to the current time minus 30 seconds. Thirty seconds is the longest time for which a page is allowed to remain dirty.
3. Determines from the per-CPU `page_state` variable the rough number of dirty pages currently in

the page cache.

4. Invokes repeatedly `writeback_inodes ( )` until either the number of pages written to disk reaches the value determined in the previous step, or all pages older than 30 seconds have been written. During this cycle the function might sleep if some request queue becomes congested.
5. Uses `mod_timer ( )` to restart the `wb_timer` dynamic timer: it will decay once again `dirty_writeback_centisecs` hundreds of seconds since the invocation of this function (or one second since now if this execution lasted too long).

15.4. The `sync( )`, `fsync( )`, and `fdatasync( )` System Calls

In this section, we examine briefly the three system calls available to user applications to flush dirty buffers to disk:

`sync( )`

Allows a process to flush all dirty buffers to disk

`fsync( )`

Allows a process to flush all blocks that belong to a specific open file to disk

`fdatasync( )`

Very similar to `fsync( )`, but doesn't flush the inode block of the file

15.4.1. The `sync( )` System Call

The service routine `sys_sync( )` of the `sync( )` system call invokes a series of auxiliary functions:

```
wakeup_bdflush(0);
sync_inodes(0);
sync_supers( );
sync_filesystems(0);
sync_filesystems(1);
sync_inodes(1);
```

As described in the previous section, `wakeup_bdflush( )` starts a *pdflush* kernel thread, which flushes to disk all dirty pages contained in the page cache.

The `sync_inodes( )` function scans the list of superblocks looking for dirty inodes to be flushed; it acts on a `wait` parameter that specifies whether it must wait until flushing has been performed or not. The function scans the superblocks of all currently mounted filesystems; for each superblock containing dirty inodes, `sync_inodes( )` first invokes `sync_sb_inodes( )` to flush the corresponding dirty pages (we described this function earlier in the section "[Looking for Dirty Pages To Be Flushed](#)"), then invokes `sync_blockdev( )` to explicitly flush the dirty buffer pages owned by the block device that includes the superblock. This is done because the `write_inode` superblock method of many disk-based filesystems simply marks the block buffer corresponding to the disk inode as dirty; the `sync_blockdev( )` function makes sure that the updates made by `sync_sb_inodes( )` are effectively written to disk.

The `sync_supers( )` function writes the dirty superblocks to disk, if necessary, by using the proper `write_super` superblock operations. Finally, the `sync_filesystems( )` executes the `sync_fs` superblock method for all writable filesystems. This method is simply a hook offered to a filesystem in case it needs to perform some peculiar operation at each sync; this method is only used by journaling filesystems such as Ext3 (see [Chapter 18](#)).

Notice that `sync_inodes ( )` and `sync_filesystems ( )` are invoked twice, once with the `wait` parameter equal to 0 and the second time with the parameter equal to 1. This is done on purpose: first, they quickly flush to disk the unlocked inodes; next, they wait for each locked inode to become unlocked and finish writing them one by one.

15.4.2. The `fsync ( )` and `fdatasync ( )` System Calls

The `fsync ( )` system call forces the kernel to write to disk all dirty buffers that belong to the file specified by the `fd` file descriptor parameter (including the buffer containing its inode, if necessary). The corresponding service routine derives the address of the file object and then invokes the `fsync` method. Usually, this method ends up invoking the `_ _writeback_single_inode ( )` function to write back both the dirty pages associated with the selected inode and the inode itself (see the section "[Looking for Dirty Pages To Be Flushed](#)" earlier in this chapter).

The `fdatasync ( )` system call is very similar to `fsync ( )`, but writes to disk only the buffers that contain the file's data, not those that contain inode information. Because Linux 2.6 does not have a specific file method for `fdatasync ( )`, this system call uses the `fsync` method and is thus identical to `fsync ( )`.